

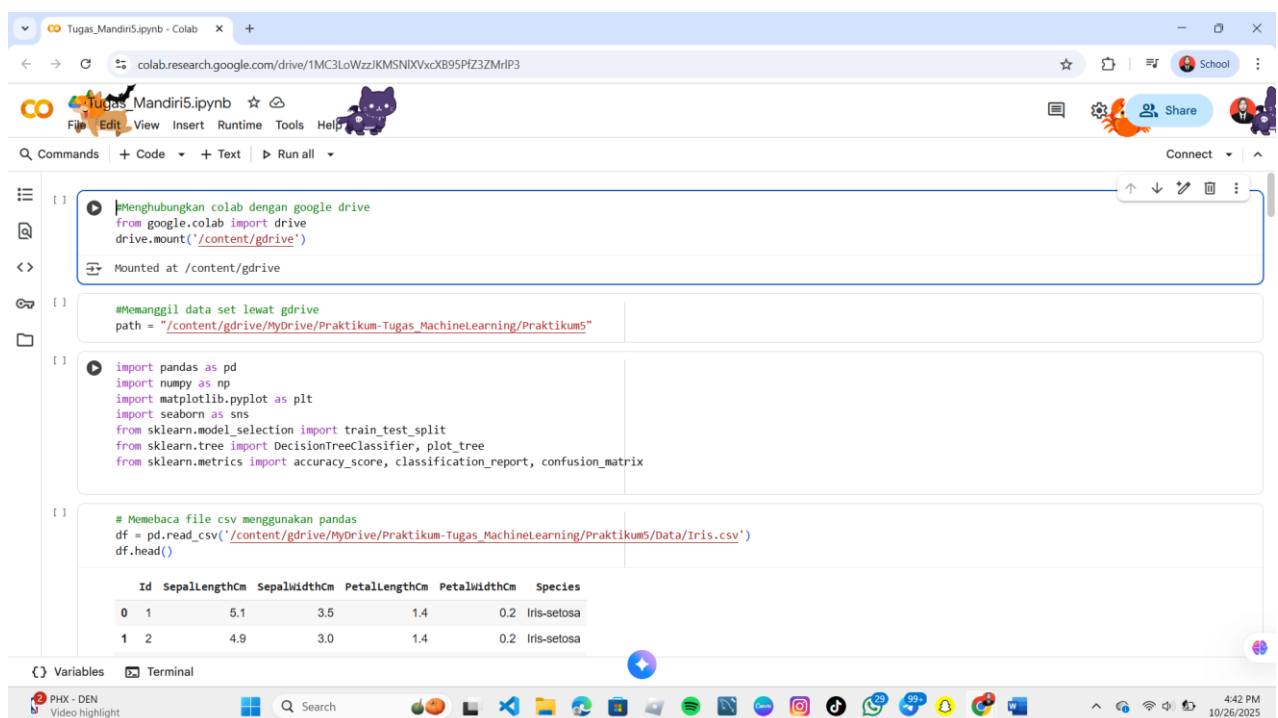
Tugas 5: Tugas Praktikum Mandiri 5 – Machine Learning

Yurida Yahsyia 1 - 0110224100 ^{1*}

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224100@student.nurulfikri.ac.id – email mahasiswa 1

1. Latihan Mandiri 5



```
[1] Menghubungkan colab dengan google drive  
from google.colab import drive  
drive.mount('/content/gdrive')  
Mounted at /content/gdrive  
[2] Memanggil data set lewat gdrive  
path = "/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum5"  
[3] import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier, plot_tree  
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  
[4] # Membaca file csv menggunakan pandas  
df = pd.read_csv('/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum5/Data/Iris.csv')  
df.head()
```

	Id	SepallengthCm	SepalwidthCm	PetallengthCm	PetalwidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa

- 1. from google.colab import drive**
Memanggil modul bawaan Colab.
- 2. drive.mount('/content/gdrive')**
Membuat “jembatan” supaya Colab bisa membaca file yang ada di Drive, dan semua file bisa diakses lewat folder /content/gdrive/MyDrive/.
- 3. path = "/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum5"**
path = alamat menuju folder Praktikum4 di dalam Google Drive.
- 4. import pandas as pd**
Mengimpor pandas sebagai pd untuk manipulasi dan analisis data (khususnya membaca dan mengolah file CSV).
- 5. import numpy as np**
Mengimpor numpy sebagai np untuk operasi numerik (terkait dengan array).
- 6. import matplotlib.pyplot as plt**

Mengimport matplotlib.pyplot sebagai plt untuk membuat visualisasi statis (grafik dan plot).

7. **import seaborn as sns**

Mengimport seaborn sebagai sns untuk membuat visualisasi statistik yang lebih menarik dan informatif, digunakan bersama matplotlib.

8. **from sklearn.model_selection import train_test_split**

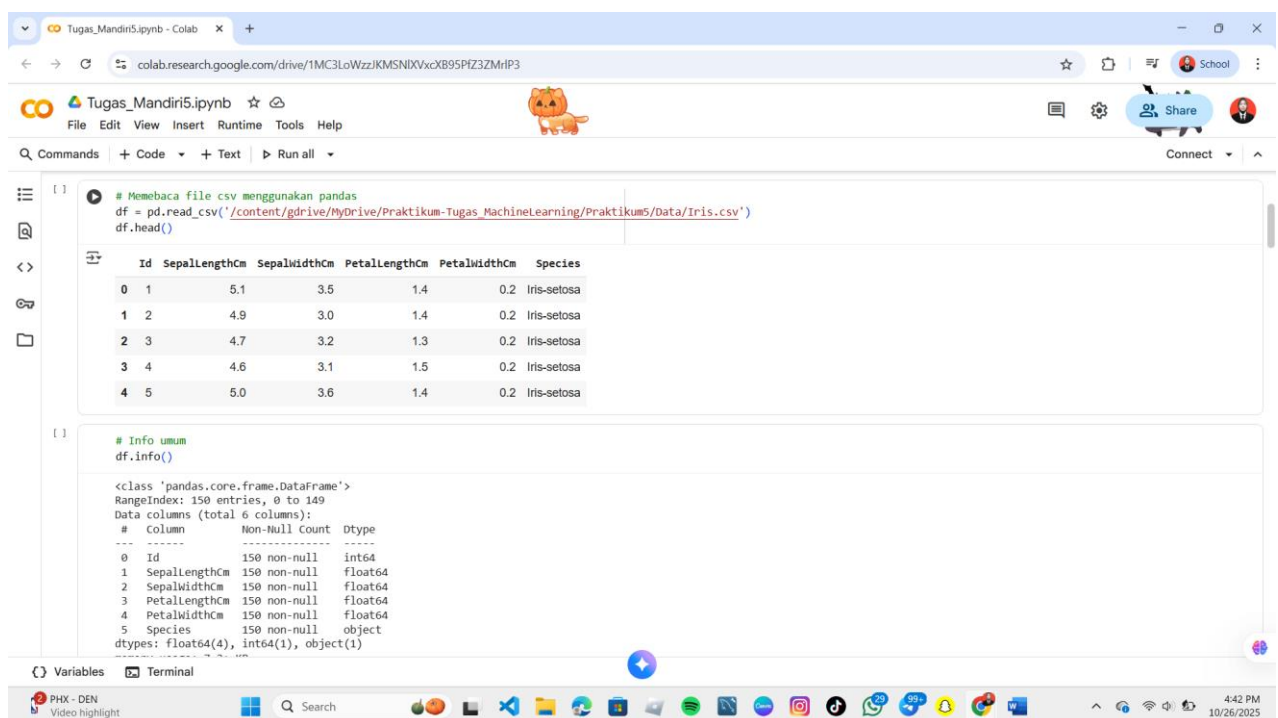
Mengimport fungsi train_test_split dari scikit-learn untuk membagi dataset menjadi set training dan testing.

9. **from sklearn.tree import DecisionTreeClassifier, plot_tree**

Mengimport kelas DecisionTreeClassifier untuk membangun model Decision Tree, dan fungsi plot_tree untuk memvisualisasikan pohon yang dihasilkan.

10. **from sklearn.metrics import accuracy_score, classification_report, confusion_matrix**

Mengimport metrik evaluasi seperti accuracy_score, classification_report, dan confusion_matrix untuk mengukur kinerja model.



```
[1] # Membaca file csv menggunakan pandas
df = pd.read_csv('/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum5/Data/Iris.csv')
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
[1] # Info umum
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   Id              150 non-null   int64  
 1   SepalLengthCm   150 non-null   float64
 2   SepalWidthCm    150 non-null   float64
 3   PetalLengthCm   150 non-null   float64
 4   PetalWidthCm    150 non-null   float64
 5   Species         150 non-null   object  
dtypes: float64(4), int64(1), object(1)
```

1. **df = pd.read_csv('/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum5/Data/Iris.csv')**

Menggunakan fungsi pd.read_csv() untuk membaca data dari lokasi file yang ditentukan di Google Drive dan menyimpannya dalam dataframe bernama df.

2. **df.head()**

Memanggil metode .head() untuk menampilkan 5 baris pertama dari dataframe df

Output:

Menampilkan 5 baris pertama dari dataset Iris.

```
# Info umum
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column        Non-Null Count  Dtype  
---  --
 0   Id            150 non-null    int64   
 1   SepalLengthCm 150 non-null    float64
 2   SepalWidthCm  150 non-null    float64
 3   PetalLengthCm 150 non-null    float64
 4   PetalWidthCm  150 non-null    float64
 5   Species        150 non-null    object  
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

1. df.info().

Output:

- Total 150 entri (baris) data.
- 6 kolom (fitur/variabel).
- Tidak ada missing value (150 non-null di semua kolom).
- Tipe data (Dtype): Id adalah integer, fitur pengukuran (SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm) adalah float (bilangan desimal), dan target klasifikasi (Species) adalah object (string/kategorikal).

```
# Statistik ringkas
print("\nStatistik deskriptif:")
display(df.describe())

Statistik deskriptif:
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  150.000000      150.000000      150.000000      150.000000      150.000000
mean    75.500000       5.843333       3.054000       3.758667       1.198667
std     43.445368       0.828066       0.433594       1.764420       0.763161
min      1.000000       4.300000       2.000000       1.000000       0.100000
25%     38.250000       5.100000       2.800000       1.600000       0.300000
50%     75.500000       5.800000       3.000000       3.350000       1.300000
75%    112.750000       6.400000       3.300000       5.100000       1.800000
max    150.000000       7.900000       4.400000       6.900000       2.500000

# Cek data kosong
print("\nJumlah data kosong tiap kolom:")
print(df.isnull().sum())

Jumlah data kosong tiap kolom:
Id                0
SepalLengthCm     0
```

1. print("\nStatistik deskriptif:")

Mencetak judul.

2. `display(df.describe())`

Menampilkan tabel Statistik Deskriptif dari dataframe `df`.

Output:

Menampilkan ringkasan data numerik (rata-rata, standar deviasi, rentang, dan kuartil) untuk fitur-fitur Iris. Ini memberikan gambaran tentang distribusi dan skala setiap fitur.



1. `print("\nJumlah data kosong tiap kolom:")`

Mencetak judul untuk output pengecekan nilai kosong.

2. `print(df.isnull().sum())`

Menghitung jumlah nilai null (kosong/hilang) untuk setiap kolom dalam dataframe `df`.

Outputnya:

Output menunjukkan semua kolom bernilai 0. Ini mengonfirmasi bahwa tidak ada missing values, sehingga data siap untuk analisis lebih lanjut.



1. `sns.pairplot(df, hue="Species", diag_kind="kde")`

Menggunakan fungsi pairplot dari library seaborn untuk membuat matriks plot.

2. `plt.suptitle("Visualisasi Dataset Iris", y=1.02)`

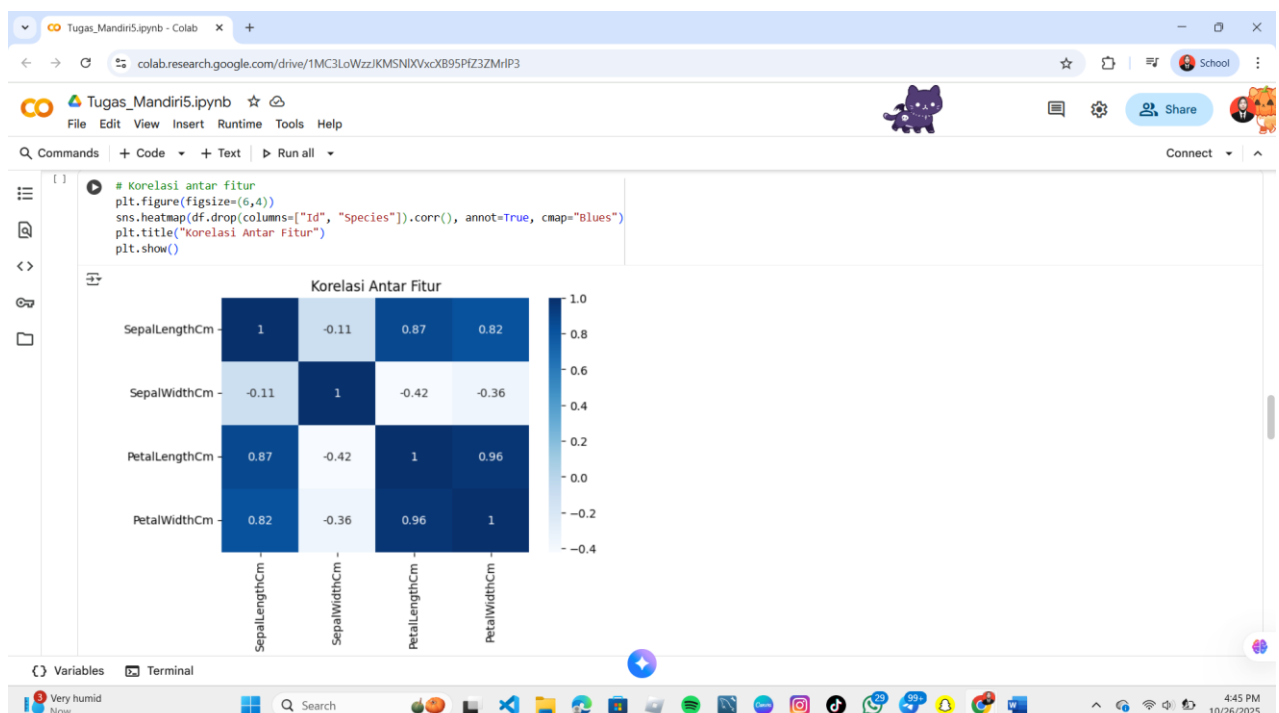
Menambahkan judul utama di atas seluruh plot.

3. `plt.show()`

Menampilkan visualisasi yang telah dibuat.

Output:

Matriks plot besar yang memvisualisasikan data



1. plt.figure(figsize=(6,4))

Mengatur ukuran figur (gambar) plot menjadi 6 inci lebar dan 4 inci tinggi.

2. sns.heatmap(df.drop(columns=["Id", "Species"]).corr(), annot=True, cmap="Blues")

Menggunakan fungsi heatmap dari library seaborn untuk memvisualisasikan matriks korelasi:

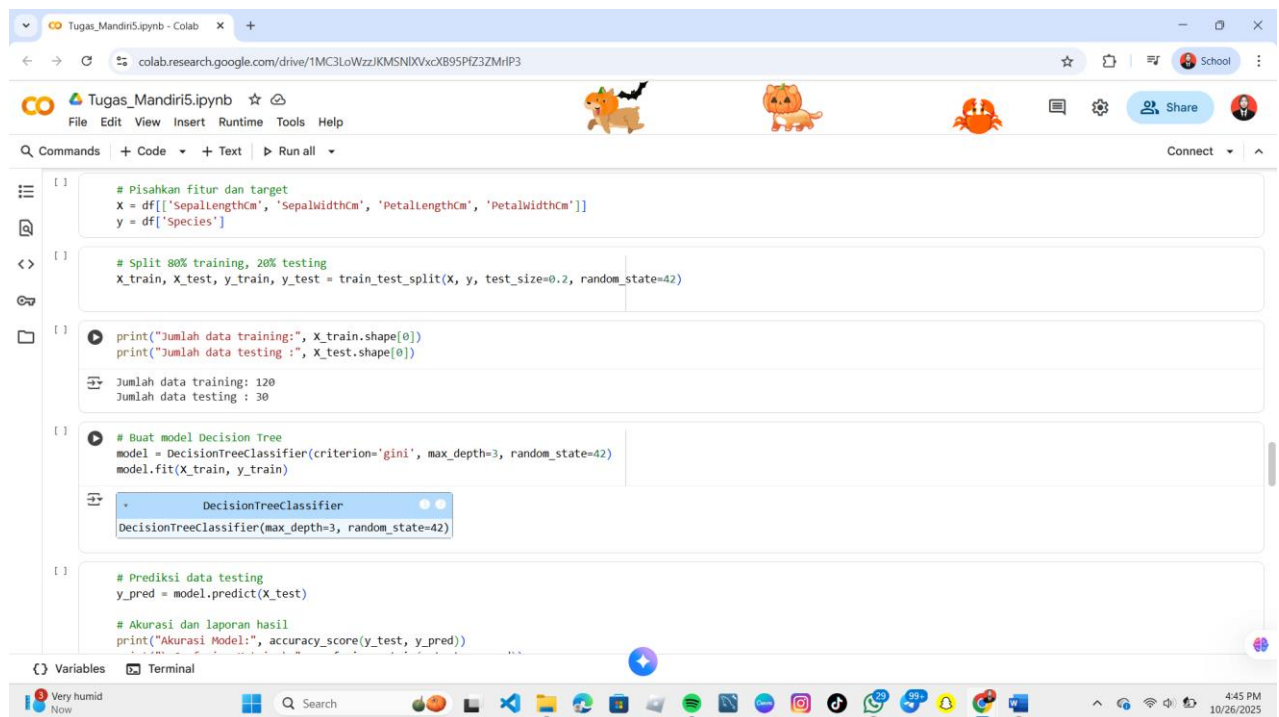
- df.drop(columns=["Id", "Species"]): Menghapus kolom Id dan Species dari dataframe.
- .corr(): Menghitung matriks korelasi (standar Pearson) dari fitur-fitur yang tersisa (SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm).
- annot=True: Menampilkan nilai korelasi (angka) di atas setiap kotak heatmap.
- cmap="Blues": Menggunakan skema warna biru; semakin gelap birunya, semakin kuat korelasi positifnya.

3. plt.title("Korelasi Antar Fitur")

Memberikan judul pada heatmap.

4. plt.show()

Menampilkan plot yang telah dibuat.



```
[ ] # Pisahkan fitur dan target
x = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = df['Species']

[ ] # Split 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

[ ] print("Jumlah data training:", X_train.shape[0])
print("Jumlah data testing :", X_test.shape[0])

Jumlah data training: 120
Jumlah data testing : 30

[ ] # Buat model Decision Tree
model = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
model.fit(X_train, y_train)

DecisionTreeClassifier(max_depth=3, random_state=42)

[ ] # Prediksi data testing
y_pred = model.predict(X_test)

# Akurasi dan laporan hasil
print("Akurasi Model:", accuracy_score(y_test, y_pred))
```

1. X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]

Mendefinisikan X sebagai dataframe yang berisi semua kolom fitur (data input) yang akan digunakan untuk membuat prediksi.

2. y = df['Species']

Mendefinisikan y sebagai series yang berisi kolom target (output) yang ingin diklasifikasikan, yaitu spesies bunga Iris.

3. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Menggunakan fungsi train_test_split dari sklearn.model_selection untuk membagi data menjadi:

- X_train dan y_train: Data 80% untuk pelatihan model.
- X_test dan y_test: Data 20% untuk pengujian model.
- test_size=0.2: Menetapkan bahwa 20% data akan digunakan untuk testing.
- random_state=42: Mengunci hasil pembagian agar selalu sama setiap kali kode dijalankan (untuk

reproduksibilitas).

4. `print("Jumlah data training:", X_train.shape[0])`

Mencetak jumlah baris (sampel) dalam set data pelatihan.

5. `print("Jumlah data testing:", X_test.shape[0])`

Mencetak jumlah baris (sampel) dalam set data pengujian.

Output:

Jumlah data training: 120 (80% dari 150)

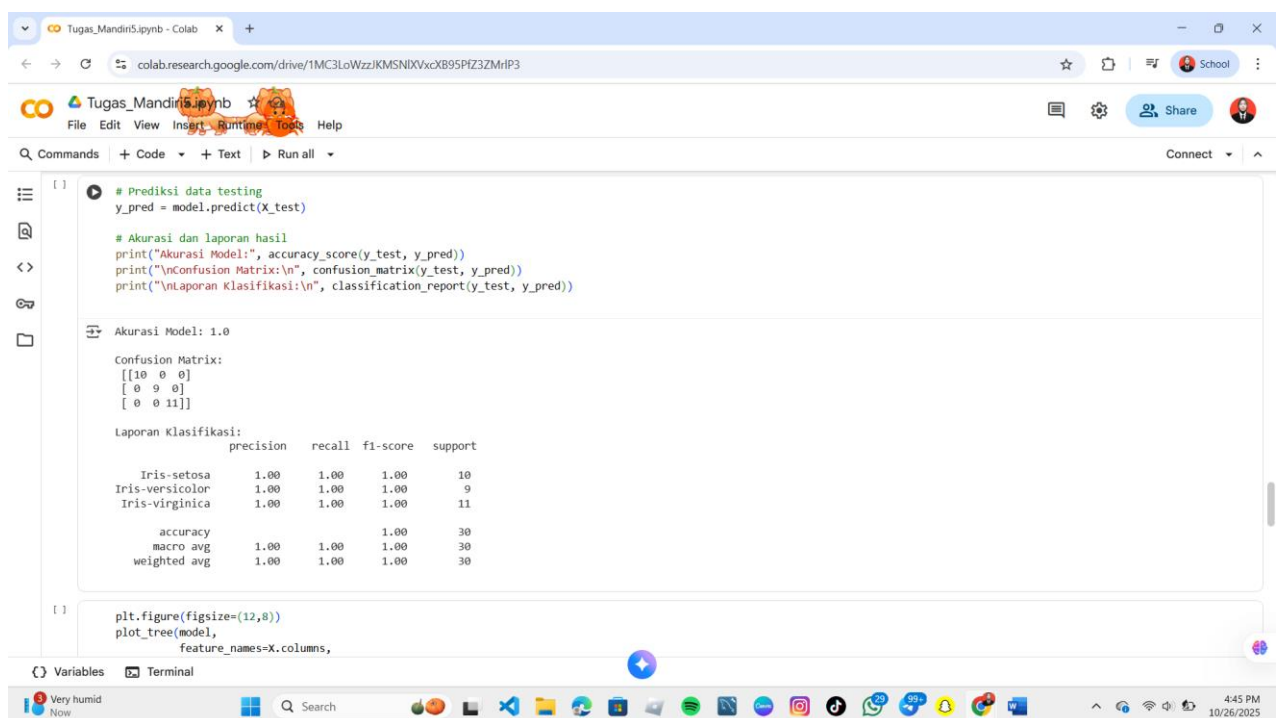
Jumlah data testing: 30 (20% dari 150)

6. `model = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)`

Menginisialisasi model Decision Tree dengan parameter

7. `model.fit(X_train, y_train)`

Melatih model Decision Tree menggunakan data 80% training (`X_train` dan `y_train`). Pada langkah ini, pohon dipotong (split) secara rekursif hingga mencapai `max_depth`.



1. `y_pred = model.predict(X_test)`

Menggunakan model yang sudah dilatih (model) untuk memprediksi label spesies (`y_pred`) dari data fitur testing (`X_test`).

2. `print("Akurasi Model:", accuracy_score(y_test, y_pred))`

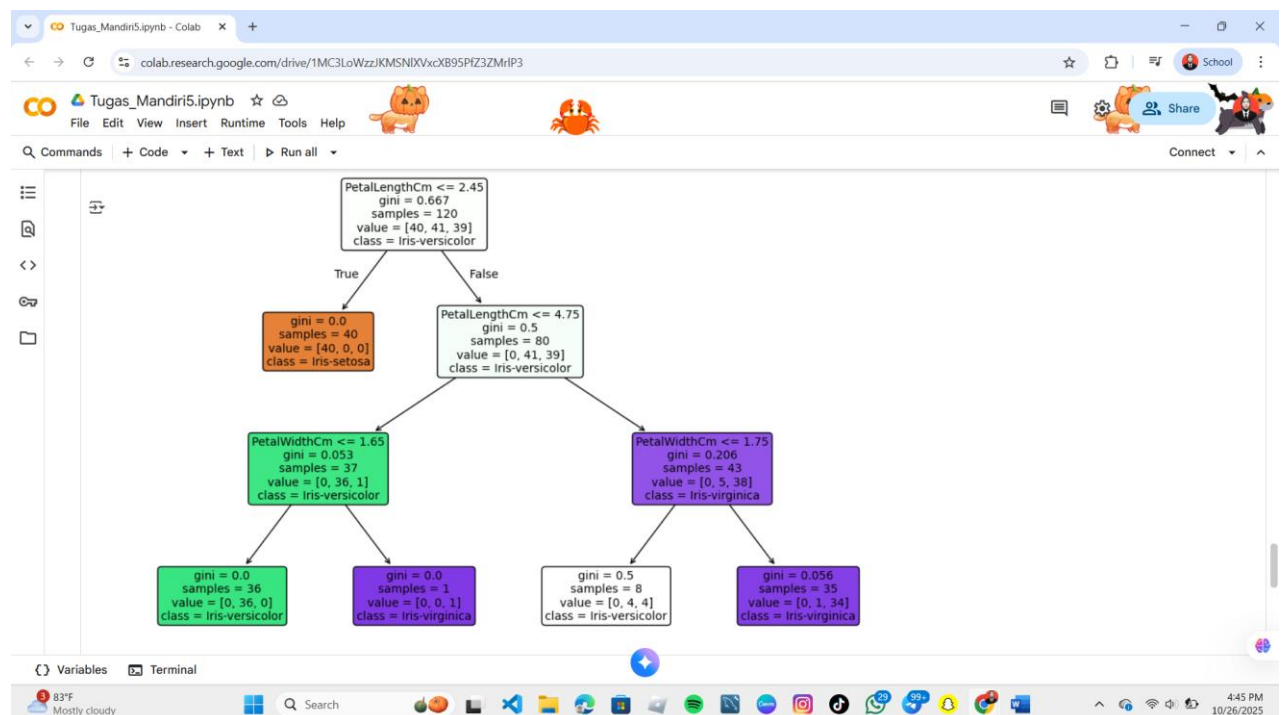
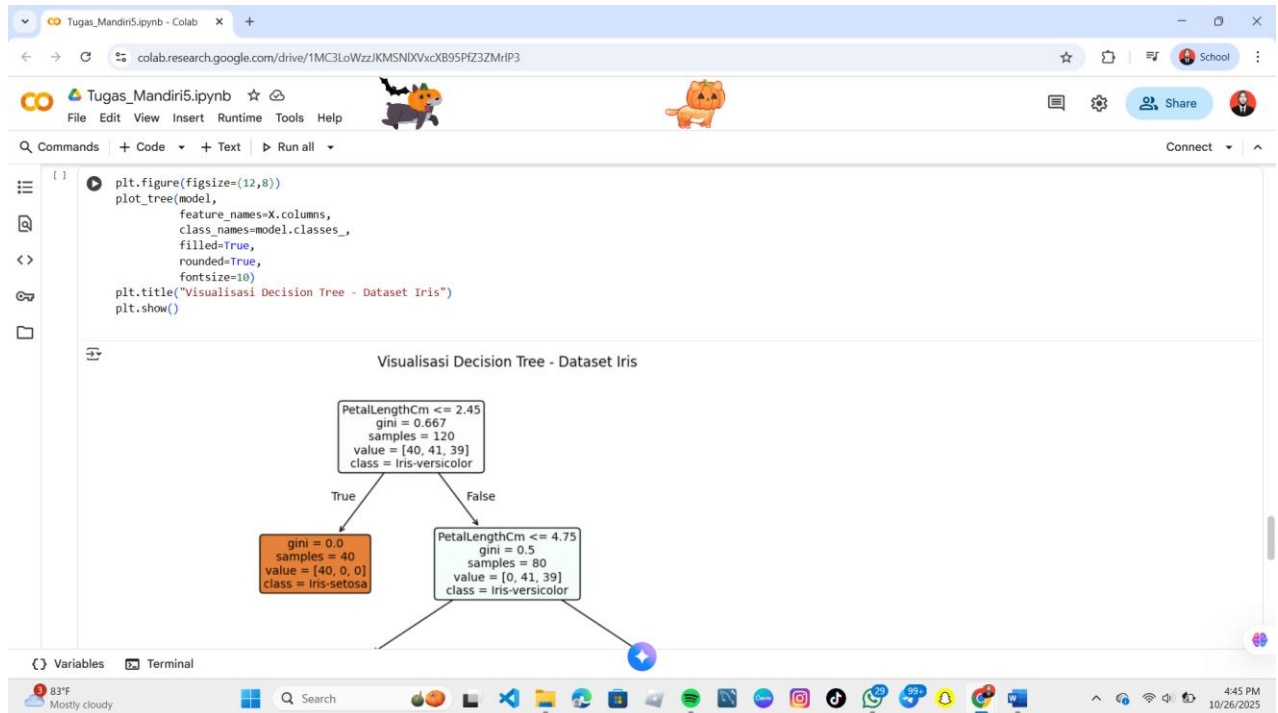
Mencetak nilai Akurasi model dengan membandingkan nilai spesies sebenarnya (`y_test`) dengan hasil prediksi (`y_pred`).

3. `print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))`

Mencetak Matriks Konfusi, tabel yang menunjukkan perbandingan antara nilai sebenarnya dan prediksi.

4. print("\nLaporan Klasifikasi:\n", classification_report(y_test, y_pred))

Mencetak Laporan Klasifikasi, yang berisi metrik precision, recall, dan f1-score untuk setiap kelas.



1. plt.figure(figsize=(12,8))

Mengatur ukuran figur plot menjadi lebar 12 inci dan tinggi 8 inci, memastikan visualisasi pohon yang kompleks terlihat jelas.

2. plot_tree(model, ...)

Fungsi inti untuk menggambar pohon.

3. plt.title("Visualisasi Decision Tree - Dataset Iris")

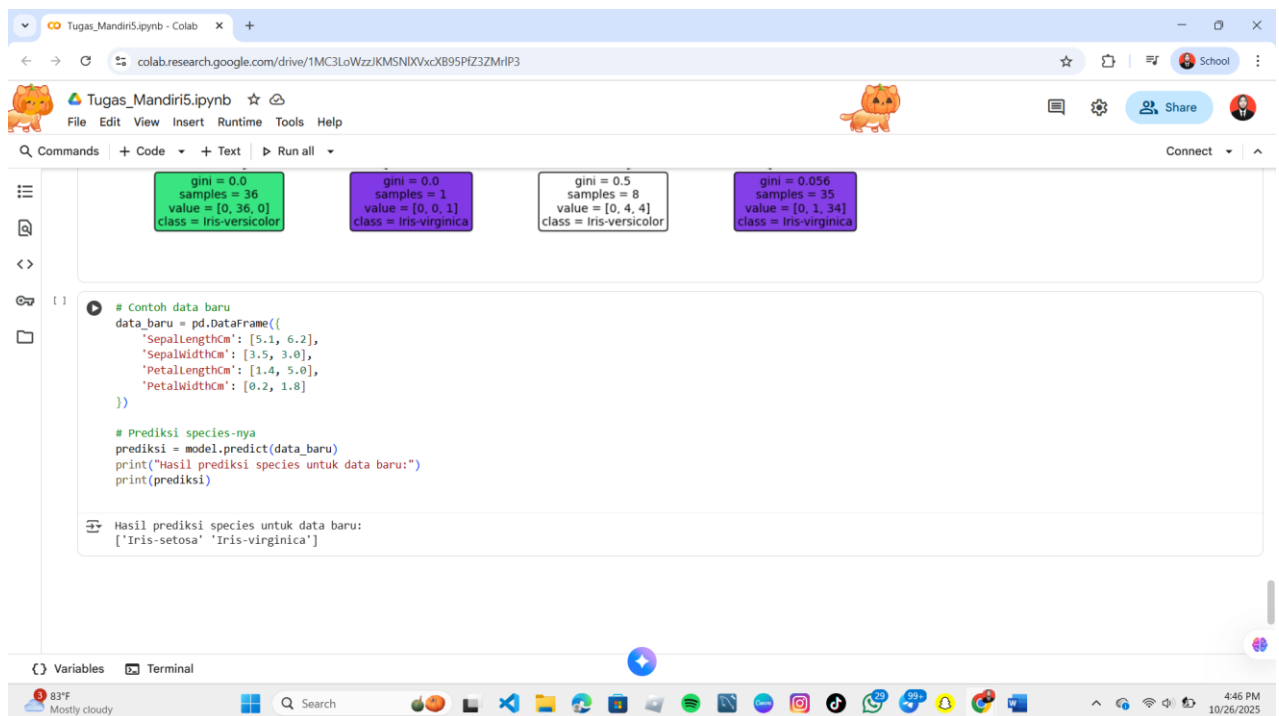
Menambahkan judul di atas plot.

4. plt.show()

Menampilkan visualisasi Decision Tree yang telah dibuat.

Outputnya:

diagram Decision Tree, yang menggambarkan jalur keputusan model, dari root node hingga leaf node.



1. data_baru = pd.DataFrame({'SepalLengthCm': [5.1, 6.2], ...})

Membuat dataframe baru bernama data_baru yang berisi dua sampel yang akan diuji.

2. prediksi = model.predict(data_baru)

Menggunakan model yang sudah dilatih (model.predict) untuk memprediksi spesies dari data_baru.

3. print("Hasil prediksi species untuk data baru:")

Mencetak judul output.

4. print(prediksi)

Mencetak hasil prediksi.

Output:

Interpretasi:

- Sampel pertama (SepalLengthCm=5.1, PetalLengthCm=1.4) diprediksi sebagai Iris-setosa.
- Sampel kedua (SepalLengthCm=6.2, PetalLengthCm=5.8) diprediksi sebagai Iris-virginica.

Hasil ini sesuai dengan jalur keputusan yang terlihat pada Decision Tree, di mana nilai PetalLengthCm yang sangat kecil pada sampel pertama langsung mengarah ke klasifikasi Iris-setosa.

Kesimpulan hasil implementasi algoritma: Berdasarkan implementasi yang telah dilakukan, model Decision Tree berhasil dilatih untuk klasifikasi spesies bunga Iris dengan kinerja yang sempurna pada data pengujian. Model mencapai 100% Akurasi pada data uji 20%, dikonfirmasi oleh Matriks Konfusi yang menunjukkan 0 kesalahan klasifikasi di antara 30 sampel, dan Precision serta Recall 1.00 untuk semua tiga spesies. Analisis visualisasi pohon, bahwa fitur PetalLengthCm (Panjang Kelopak) adalah pemisah paling dominan yang digunakan model untuk mencapai kemurnian kelas, sementara visualisasi data awal (Pair Plot) memperkuat temuan ini dengan menunjukkan batas pemisahan yang sangat jelas antar spesies berdasarkan fitur-fitur kelopak.

LINK GITHUB UPLOAD TUGAS : <https://github.com/Yurida26/Machine-Learning/tree/ff0470d81544532b466440e73c0cad36e1af4209/Praktikum5>

